VE280-26SP Project 2

Background

In this project you will implement two classic recursive data structures - **linked lists** and **binary trees** - entirely from scratch, without any provided helper code. You must define the node structs, write constructor and destructor functions, and implement a set of recursive algorithms on each structure. Finally, you will write a `main.cpp` driver that reads commands from standard input and dispatches them to your functions, enabling automated testing via `.in` / `.out` files.

File	Description
`p2.h`	Defines `ListNode`, `TreeNode`, and declares all functions listed in this document
`p2.cpp`	Implements every function declared in `p2.h`
`main.cpp`	Command-dispatch driver; described in the [Input Format](#) section

Do **not** submit any other files. Every function you implement in `p2.cpp` **must be recursive**. Loop constructs (`for`, `while`, `do-while`, `goto`) are **not permitted** anywhere in `p2.cpp`. You may only `#include <iostream>` and `#include <cstdlib>` in `p2.cpp`.

Part 1 - Linked Lists

1.1 Data Structure Definition

A **linked list** is a chain of nodes, each holding one integer value and a pointer to the next node. In `p2.h`, define the following struct exactly:

```
struct ListNode {
  int val;
  ListNode *next;
};
```

The **empty list** is represented by `nullptr`. A list `( v1 v2 ... vn )` is a chain where the first node holds `v1`, its `next` field points to a node holding `v2`, and so on, with the last node's `next` set to `nullptr`.

For example, `( 1 2 3 )` is stored as:

```
[1] -> [2] -> [3] -> nullptr
```

1.2 Functions to Implement

`ListNode *list_construct(const int *arr, int n)`

- **Requires:** `arr` points to an array of at least `n` integers. `n >= 0`.

- **Effects:** Allocates and returns a new linked list containing `arr[0], arr[1], ..., arr[n-1]` in that order. Returns `nullptr` if `n == 0`.
- **Must** be implemented recursively (no loops).

`void list_destroy(ListNode *list)`

- **Effects:** Recursively deallocates every node reachable from `list`. Does nothing if `list` is `nullptr`.

`void list_print(const ListNode *list)`

- **Effects:** Prints `list` to `cout` as `( v1 v2 ... vn )` followed by a newline. Prints `( )` followed by a newline if `list` is `nullptr`.
- **Must** be implemented recursively (no loops).
- **Example:** A list `( 1 2 3 )` produces the output:

```
( 1 2 3 )
```

`int size(const ListNode *list)`

- **Effects:** Returns the number of elements in `list`. Returns `0` if `list` is `nullptr`.

`int sum(const ListNode *list)`

- **Effects:** Returns the sum of all elements in `list`. Returns `0` if `list` is `nullptr`.

`int product(const ListNode *list)`

- **Effects:** Returns the product of all elements in `list`. Returns `1` if `list` is `nullptr`.

`ListNode *reverse(const ListNode *list)`

- **Effects:** Allocates and returns a new list that is the reverse of `list`. The original `list` is not modified.
- **Example:** `reverse` of `( 3 2 1 )` returns a new list `( 1 2 3 )`.

`ListNode *append(const ListNode *first, const ListNode *second)`

- **Effects:** Allocates and returns a new list consisting of all elements of `first` followed by all elements of `second`. Neither `first` nor `second` is modified.
- **Example:** `append(( 2 4 6 ), ( 1 3 ))` returns a new list `( 2 4 6 1 3 )`.

`ListNode *filter_odd(const ListNode *list)`

- **Effects:** Allocates and returns a new list containing only the elements of `list` that are odd, in their original order. The original `list` is not modified.
- **Example:** `filter_odd(( 3 4 1 5 6 ))` returns a new list `( 3 1 5 )`.

`ListNode *filter_even(const ListNode *list)`

- **Effects:** Allocates and returns a new list containing only the elements of `list` that are even, in their original order. The original `list` is not modified.

`ListNode *filter(const ListNode *list, bool (*fn)(int))`

- **Effects:** Allocates and returns a new list containing precisely the elements of `list` for which `fn(element)` returns `true`, in their original order. The original `list` is not modified.
- **Example:** If `fn` returns `true` for odd integers, `filter(list, fn)` produces the same result as `filter_odd(list)`.

`ListNode *insert_list(const ListNode *first, const ListNode *second, unsigned int n)`

- **Requires:** `n <= size(first)`.
- **Effects:** Allocates and returns a new list consisting of:
  1. The first `n` elements of `first`,
  2. followed by all elements of `second`,
  3. followed by the elements of `first` starting at position `n` (0-indexed).

  Neither `first` nor `second` is modified.
- **Example:** `insert_list(( 1 2 3 ), ( 4 5 6 ), 2)` returns `( 1 2 4 5 6 3 )`.

`ListNode *chop(const ListNode *list, unsigned int n)`

- **Requires:** `size(list) >= n`.
- **Effects:** Allocates and returns a new list equal to `list` with its last `n` elements removed. The original `list` is not modified.
- **Example:** `chop(( 1 2 3 4 5 ), 2)` returns `( 1 2 3 )`.

Part 2 - Binary Trees

2.1 Data Structure Definition

A **binary tree** is either empty (`nullptr`) or a node holding one integer value with a left subtree and a right subtree. In `p2.h`, define the following struct exactly:

```
struct TreeNode {
  int val;
  TreeNode *left;
  TreeNode *right;
};
```

A node with both children set to `nullptr` is called a **leaf**. The **empty tree** is `nullptr`.

2.2 Functions to Implement

`TreeNode *tree_construct(const int *preorder, const int *inorder, int n)`

- **Requires:**
  - `preorder` and `inorder` are arrays of length `n` containing the **same** `n` **distinct** integers.
  - `preorder` gives the preorder traversal (root, left subtree, right subtree) of a binary tree.
  - `inorder` gives the inorder traversal (left subtree, root, right subtree) of the same tree.
  - `n >= 0`.
- **Effects:** Allocates and returns the unique binary tree whose preorder and inorder traversals match the given arrays. Returns `nullptr` if `n == 0`.
- **Must** be implemented recursively (no loops).
- **Algorithm hint:** The first element of `preorder` is always the root. Find that value in `inorder`; elements to its left belong to the left subtree, elements to its right belong to the right subtree. Recurse on each part.
- **Example:** Given `preorder = [4, 2, 3, 5]` and `inorder = [2, 3, 4, 5]`, `tree_construct` builds:

```
    4
   / \
  2   5
   \
    3
```

`void tree_destroy(TreeNode *tree)`

- **Effects:** Recursively deallocates every node reachable from `tree`. Does nothing if `tree` is `nullptr`.

`void tree_print(const TreeNode *tree)`

- **Effects:** Prints the tree to `cout` in a **rotated in-order layout**:

- Traverse in the order: right subtree, root, left subtree.
- Each node is printed on its own line, indented by `4 * depth` spaces (the root is at depth 0, its children at depth 1, and so on).
- Each line contains only the spaces and the integer value, followed by a newline.
- Prints nothing if `tree` is `nullptr`.

- **Must** be implemented recursively (no loops).
- **Example:** For the tree:

```
    4
   / \
  2   5
   \
    3
```

`tree_print` produces:

```
    5
4
        3
    2
```

Explanation: `5` is at depth 1 (4 spaces), `4` is at depth 0 (0 spaces), `3` is at depth 2 (8 spaces), `2` is at depth 1 (4 spaces). Reading top-to-bottom gives the reverse in-order sequence `5 4 3 2`.

`int tree_sum(const TreeNode *tree)`

- **Effects:** Returns the sum of all node values in `tree`. Returns `0` if `tree` is `nullptr`.

`bool tree_search(const TreeNode *tree, int key)`

- **Effects:** Returns `true` if any node in `tree` holds value `key`; returns `false` otherwise. Returns `false` if `tree` is `nullptr`.

`int depth(const TreeNode *tree)`

- **Effects:** Returns the number of nodes on the longest path from the root to any leaf. Returns `0` if `tree` is `nullptr`. A single-node tree has depth `1`.
- **Example:**

```
        4
       / \
      2   5
       \   \
        3   7
       /
      6
```

This tree has depth `4`.

`int tree_min(const TreeNode *tree)`

- **Requires:** `tree != nullptr`.
- **Effects:** Returns the minimum value among all nodes in `tree`.

`ListNode *traversal(const TreeNode *tree)`

- **Effects:** Allocates and returns a new list containing the node values of `tree` in **inorder** (left subtree $\rightarrow$ root $\rightarrow$ right subtree). Returns `nullptr` if `tree` is `nullptr`.
- **Example:** For the tree:

```
    4
   / \
  2   5
   \
    3
```

`traversal` returns `( 2 3 4 5 )`.

`bool tree_hasPathSum(const TreeNode *tree, int target)`

- **Effects:** Returns `true` if and only if `tree` contains at least one root-to-leaf path whose node values sum to `target`. Returns `false` if `tree` is `nullptr`.
- **Definition:** A **root-to-leaf path** is a sequence of nodes beginning at the root and ending at a leaf (a node whose both children are `nullptr`).
- **Example:**

```
      4
     / \
    1   5
   / \
  3   6
```

The three root-to-leaf paths and their sums are: `4->1->3` = 8, `4->1->6` = 11, `4->5` = 9. `tree_hasPathSum(tree, 9)` returns `true`; `tree_hasPathSum(tree, 10)` returns `false`.

2.3 The "Covered By" Relation

Define the binary relation **"A is covered by B"** on trees recursively:

- `nullptr` is covered by **every** tree (including `nullptr`).
- A non-empty tree A is covered by tree B if and only if:
    1. B is non-empty,
    2. the root values are equal: `A->val == B->val`,
    3. `A->left` is covered by `B->left`, and
    4. `A->right` is covered by `B->right`.

Note that `nullptr` covers **only** `nullptr` (if A is non-empty, it is not covered by `nullptr`).

Example. Consider the following two trees:

```
     4                4
    / \              / \
   2   5    vs.     2   5
    \
     3
```

The left tree is **not** covered by the right tree (because the left tree has a node 3 that the right tree lacks). The right tree **is** covered by the left tree.

`bool covered_by(const TreeNode *A, const TreeNode *B)`

- **Effects:** Returns `true` if tree A is covered by tree B according to the definition above; `false` otherwise.

2.4 The "Contained By" Relation

Tree A is **contained by** tree B if:

- A is covered by B, **or**
- A is covered by the left subtree of B, **or**
- A is covered by the right subtree of B, (recursively).

More precisely, A is contained by B if A is covered by **some subtree** of B. A **subtree** of T is any node S of T together with all of S's descendants; the empty tree `nullptr` is also considered a subtree of every tree.

Example. For tree T:

```
        4
       / \
      2   5
       \   \
        3   8
       / \
      6   7
```

The following trees **are** contained by T:

```
    4              2              5
   / \              \              \
  2   5              3              8
                    / \
                   6   7
```

The following tree is **not** contained by T (no node in T with value 4 has left child 3):

```
    4
   / \
  3   5
```

`bool contained_by(const TreeNode *A, const TreeNode *B)`

- **Effects:** Returns `true` if tree A is covered by tree B or by any subtree of B; `false` otherwise.

2.5 Sorted Binary Trees and Insertion

A **sorted binary tree** is a binary tree in which:

- Every value in the **left** subtree is **strictly less than** the root value.
- Every value in the **right** subtree is **greater than or equal to** the root value.
- Both subtrees are themselves sorted binary trees.
- `nullptr` is a valid sorted binary tree.

Valid examples:

```
  4              4              4              4
 / \            / \            / \            / \
2   5          2   6          2   5          2   5
              / \ / \          \            / \
             1  3 5  7          3          1   3
```

Invalid examples:

```
  1                1                4
 /               / \              / \
1               2   3            3   6
                                      \
                                       1
```

- First: left child `1` equals root `1`; left subtree values must be **strictly less**.
- Second: left child `2 > 1`; left subtree values must be strictly less than root.
- Third: node `1` is in the right subtree of `6`, but `1 < 6`; right subtree values must be `>= 6`.

`TreeNode *insert_tree(int elt, const TreeNode *tree)`

- **Requires:** `tree` is a valid sorted binary tree.
- **Effects:** Allocates and returns a new sorted binary tree identical to `tree` with `elt` inserted at the unique leaf position that maintains the sorted property. The original `tree` is not modified.
- **Example:** Inserting `1` into:

  ```
      4
     / \
    2   5
     \
      3
  ```

  yields:

  ```
      4
     / \
    2   5
   / \
  1   3
  ```

- **Hint:** There is exactly one valid position for any inserted element.

Input Format

Write `main.cpp` to maintain a named store of lists and trees. It reads commands from **standard input**, one command per line, until EOF. For each command, it calls the appropriate function(s) from `p2.h` / `p2.cpp` and writes the required output to **standard output**.

Memory management rules:

- A stored object persists until it is explicitly removed by `LIST_DEL` / `TREE_DEL`, or replaced when a new command writes to a name that already exists.
- When a name is overwritten, the **old** object must be freed immediately before the new one is stored.
- When the program reaches EOF, every object still in the store must be freed before the program

exits.

- Any intermediate allocation made during a command that is not stored (e.g., a temporary result immediately discarded) must also be freed.

Token conventions:

- `<name>`: an arbitrary non-whitespace string used to identify a stored list or tree.
- `<int>`: a decimal integer (may be negative).
- `<n>`: a non-negative decimal integer.
- `<arr>`: `n` space-separated integers on the same line as the command.
- All referenced names are guaranteed to exist at the time the command is issued, unless the command creates a new entry.
- If a destination name already exists, the old object is replaced and its memory is freed.

List Commands

Command	Input line format	Description
`LIST_NEW <name> <n> <arr>`	`LIST_NEW name n v1 v2 ... vn`	Constructs a list from the `n` integers `v1..vn` using `list_construct` and stores it as `name`.
`LIST_DEL <name>`	`LIST_DEL name`	Calls `list_destroy` on the stored list and removes it.
`LIST_PRINT <name>`	`LIST_PRINT name`	Calls `list_print` on the stored list.
`LIST_SIZE <name>`	`LIST_SIZE name`	Prints `size(list)` followed by a newline.
`LIST_SUM <name>`	`LIST_SUM name`	Prints `sum(list)` followed by a newline.
`LIST_PRODUCT <name>`	`LIST_PRODUCT name`	Prints `product(list)` followed by a newline.
`LIST_REVERSE <dst> <src>`	`LIST_REVERSE dst src`	Stores `reverse(src)` as `dst`.
`LIST_APPEND <dst> <src1> <src2>`	`LIST_APPEND dst src1 src2`	Stores `append(src1, src2)` as `dst`.
`LIST_FILTER_ODD <dst> <src>`	`LIST_FILTER_ODD dst src`	Stores `filter_odd(src)` as `dst`.
`LIST_FILTER_EVEN <dst> <src>`	`LIST_FILTER_EVEN dst src`	Stores `filter_even(src)` as `dst`.
`LIST_FILTER <dst> <src> <pred>`	`LIST_FILTER dst src ODD` or `LIST_FILTER dst src EVEN`	Stores `filter(src, fn)` as `dst`, where `fn` is the predicate corresponding to `ODD` (returns `true` for odd integers) or `EVEN` (returns `true` for even integers).
`LIST_INSERT <dst> <src1> <src2> <n>`	`LIST_INSERT dst src1 src2 n`	Stores `insert_list(src1, src2, n)` as `dst`.
`LIST_CHOP <dst> <src> <n>`	`LIST_CHOP dst src n`	Stores `chop(src, n)` as `dst`.

Tree Commands

Command	Input line format	Description
`TREE_NEW <name> <n> <pre> <in>`	`TREE_NEW name n p1..pn i1..in`	Constructs a tree from `n` preorder values `p1..pn` followed by `n` inorder values `i1..in` using `tree_construct`, stores as `name`. If `n == 0` the tree is empty.
`TREE_DEL <name>`	`TREE_DEL name`	Calls `tree_destroy` on the stored tree and removes it.
`TREE_PRINT <name>`	`TREE_PRINT name`	Calls `tree_print` on the stored tree.
`TREE_SUM <name>`	`TREE_SUM name`	Prints `tree_sum(tree)` followed by a newline.
`TREE_SEARCH <name> <key>`	`TREE_SEARCH name key`	Prints `true` if `tree_search(tree, key)`, `false` otherwise, followed by a newline.
`TREE_DEPTH <name>`	`TREE_DEPTH name`	Prints `depth(tree)` followed by a newline.
`TREE_MIN <name>`	`TREE_MIN name`	Prints `tree_min(tree)` followed by a newline. Tree is guaranteed non-empty.
`TREE_TRAVERSAL <dst> <src>`	`TREE_TRAVERSAL dst src`	Stores `traversal(src)` as list `dst`.
`TREE_PATHSUM <name> <target>`	`TREE_PATHSUM name target`	Prints `true` if `tree_hasPathSum(tree, target)`, `false` otherwise, followed by a newline.
`TREE_INSERT <dst> <src> <elt>`	`TREE_INSERT dst src elt`	Stores `insert_tree(elt, src)` as `dst`.
`TREE_COVERED <nameA> <nameB>`	`TREE_COVERED A B`	Prints `true` if `covered_by(A, B)`, `false` otherwise, followed by a newline.
`TREE_CONTAINED <nameA> <nameB>`	`TREE_CONTAINED A B`	Prints `true` if `contained_by(A, B)`, `false` otherwise, followed by a newline.

Sample Input

```
LIST_NEW a 5 1 2 3 4 5
LIST_PRINT a
LIST_SIZE a
LIST_SUM a
LIST_PRODUCT a
LIST_NEW b 3 6 7 8
LIST_APPEND c a b
```

```
LIST_PRINT c
LIST_REVERSE d a
LIST_PRINT d
LIST_FILTER_ODD e a
LIST_PRINT e
LIST_FILTER_EVEN j a
LIST_PRINT j
LIST_DEL j
LIST_FILTER h a ODD
LIST_PRINT h
LIST_INSERT f a b 2
LIST_PRINT f
LIST_CHOP g a 2
LIST_PRINT g
TREE_NEW t 4 4 2 3 5 2 3 4 5
TREE_PRINT t
TREE_SUM t
TREE_DEPTH t
TREE_MIN t
TREE_SEARCH t 3
TREE_PATHSUM t 9
TREE_TRAVERSAL tinorder t
LIST_PRINT tinorder
TREE_COVERED t t
TREE_CONTAINED t t
TREE_INSERT u t 1
TREE_PRINT u
TREE_DEL u
```

Sample Output

```
( 1 2 3 4 5 )
5
15
120
( 1 2 3 4 5 6 7 8 )
( 5 4 3 2 1 )
( 1 3 5 )
( 2 4 )
( 1 3 5 )
( 1 2 6 7 8 3 4 5 )
( 1 2 3 )
    5
4
        3
    2
14
3
2
true
true
```

```
( 2 3 4 5 )
true
true
    5
4
        3
    2
        1
```

Requirements

- Implement all functions in `p2.cpp` and write the driver in `main.cpp`.
- Every function must be implemented **recursively** with no loops.
- All dynamically allocated memory must be managed correctly: free replaced objects immediately on overwrite, free all remaining stored objects at program exit, and free any intermediate allocations that are not stored.

Submission

Put your `p2.h`, `p2.cpp`, and `main.cpp` directly in your personal repository as `p2/p2.h`, `p2/p2.cpp`, and `p2/main.cpp`, push and release. The scope of the commit message should be `p2`, and both the release tag and release name should be `p2`.

JOJ will open between **2026-03-13 00:05:00+08:00** and **2026-03-27 00:05:00+08:00**.